

RESEARCH METHODOLOGY DOCUMENT

Invisible Playmaker Index (IPI) Detailed Methodology

This document provides a complete, step-by-step explanation of how the **Invisible Playmaker Index (IPI)** works — what each component measures, why it is needed, and how to implement it in Python. The IPI addresses the "FIFA Paradox": the systematic failure of traditional stats to credit structural contributions like deep passes, team organization, and space creation.

The FIFA Paradox — Why We Need IPI

Traditional stats (xG, xA) only reward **TERMINAL** actions: goals and assists.
A brilliant through-ball by Laporte that sets up a goal gets **ZERO** credit.
Deep midfielders like Rodri and Busquets are perpetually undervalued.
IPI fixes this by measuring the **VALUE** of every single action on the pitch.

The IPI Formula

$$\begin{aligned} \text{IPI} = & 0.6 \times \text{ActionValue} \\ & + 0.2 \times \text{EntropyReduction} \\ & + 0.2 \times \text{DefensiveDisruption} \end{aligned}$$

A player's total IPI score = sum of IPI over all their actions in a match.
Weights reflect the relative importance and reliability of each component.

Document Contents

Part 1	ActionValue (60%) Measures how much each pass or carry increases goal probability using an xT grid.
Part 2	EntropyReduction (20%) Measures whether a player's action makes the team more structurally organized.
Part 3	DefensiveDisruption (20%) Measures whether an action forces the opposing defense to spread and create gaps.

Part 4	Combining into IPI Normalization, weighting, aggregation, and producing final player rankings.
Part 5	4-Week Implementation Plan Week-by-week build plan with tools, libraries, and deliverables.
Part 6	Validation Three methods for confirming the model surfaces genuinely undervalued players.
Part 7	Related Work & References VAEP, xT, and other foundational papers that inspired this work.

PART 1 — ActionValue (Weight: 60%)

Objective: Quantify how much each pass or carry **increased the probability of scoring** by measuring the change in Expected Threat (xT) between start and end locations.

Why We Need It

Traditional stats reward only the final action before a goal — the shot, the last pass. A midfielder who plays a decisive line-breaking ball through a defensive block gets zero statistical credit under xG or xA. ActionValue assigns explicit credit to **every ball-progression action** based on how much it moved the ball into a more dangerous area of the pitch.

Laporte Case Study

Laporte plays a through-ball from midfield (xT = 0.03) into the opponent box edge (xT = 0.18).

ActionValue = $0.18 - 0.03 = +0.15$

Under traditional stats: 0 credit. Under ActionValue: significant contribution recorded.

How to Implement It

Step 1 — Build the Pitch Grid

Divide the 120m x 80m pitch into a **12 x 8 grid of 96 cells**, each roughly 10m x 10m. Every event on the pitch maps to exactly one cell.

```
import numpy as np

# Pitch dimensions: 120m x 80m | Grid: 12 cols x 8 rows
GRID_X, GRID_Y = 12, 8
cell_w = 120 / GRID_X  # = 10.0 m per cell (horizontal)
cell_h = 80 / GRID_Y    # = 10.0 m per cell (vertical)

def get_cell(x, y):
    cx = min(int(x / cell_w), GRID_X - 1)
    cy = min(int(y / cell_h), GRID_Y - 1)
    return cx, cy
```

Step 2 — Compute xT Values from Historical Data

For each cell, calculate: *"If a team has possession here, what is the probability a goal results from this possession?"* Use StatsBomb historical match data to count goals vs. total possessions through each

cell.

```
# Initialize arrays
xT = np.zeros((GRID_X, GRID_Y))
count = np.zeros((GRID_X, GRID_Y))

# Loop through all historical possessions
for possession in historical_data:
    goal_scored = possession.resulted_in_goal # 1 or 0
    for event in possession.events:
        cx, cy = get_cell(event.x, event.y)
        xT[cx, cy] += goal_scored
        count[cx, cy] += 1

# Normalize: xT[cell] = P(goal | possession in cell)
xT = xT / (count + 1e-6)

# Typical values after normalization:
# Cells in opponent box edge -> xT ~ 0.15 to 0.25
# Centre circle -> xT ~ 0.02 to 0.04
# Own defensive half -> xT ~ 0.01 to 0.02
```

Step 3 — Calculate ActionValue for Each Event

For every pass or carry in a match, compute the difference in xT between the destination cell and the origin cell.

```
def action_value(start_x, start_y, end_x, end_y, xT_grid):
    start_cell = get_cell(start_x, start_y)
    end_cell    = get_cell(end_x,    end_y)
    av = float(xT_grid[end_cell]) - float(xT_grid[start_cell])
    return av

# Example 1 – Laporte through-ball:
# start (50, 40) -> cell xT = 0.03
# end   (97, 38) -> cell xT = 0.18
# ActionValue = +0.15    <- large positive: ball moved into danger zone

# Example 2 – Backward safety pass under pressure:
# start (60, 30) -> cell xT = 0.05
# end   (38, 28) -> cell xT = 0.02
# ActionValue = -0.03    <- small negative: ball moved away from danger

# Example 3 – Lateral pass same line:
# start (55, 20) -> cell xT = 0.04
# end   (55, 60) -> cell xT = 0.04
# ActionValue = 0.00     <- neutral: no threat change
```

Key Insight

ActionValue is SIGNED: positive = ball moved toward danger (good), negative = ball moved away from danger, zero = lateral/neutral action. Summing over a match gives each player a net ball-progression score.

PART 2 — EntropyReduction (Weight: 20%)

Objective: Measure whether a player's action makes their team **more organized and compact** (lower entropy) or more scattered and disorganized (higher entropy).

Why We Need It

The greatest contribution of players like Busquets, Rodri, or Pirlo is not moving the ball into dangerous zones — it is maintaining team **structural cohesion**. They position themselves and distribute the ball in ways that keep teammates well-spaced and organized. This has zero representation in any traditional metric. EntropyReduction captures it by measuring team shape using **Shannon Entropy** on the spatial density of player positions.

Shannon Entropy from information theory measures how 'spread out' or 'uncertain' a distribution is. Applied to player positions: **high entropy = players scattered across the pitch** (disorganized), **low entropy = players clustered and compact** (structured). A positive EntropyReduction means the action caused the team to get more organized.

How to Implement It

Step 1 — Extract Teammate Positions from Freeze-Frame

StatsBomb 360 data provides the (x, y) positions of all 22 players at the moment of every event. We extract the 10 outfield teammates of the player who performed the action (excluding the ball carrier and goalkeeper).

```
import numpy as np

# StatsBomb freeze_frame: list of dicts
# Each dict has: 'location': [x, y], 'teammate': True/False, 'actor': True/False

def get_teammate_positions(freeze_frame):
    positions = []
    for player in freeze_frame:
        # Include teammates but exclude the ball carrier (actor)
        if player['teammate'] and not player.get('actor', False):
            positions.append(player['location'])
    return np.array(positions)  # shape: (10, 2)
```

Step 2 — Estimate Team Shape using Kernel Density Estimation (KDE)

KDE takes the 10 discrete player positions and produces a **smooth probability density function** over the entire pitch. Think of it as a heat map showing where the team is 'concentrated'. A compact team produces a narrow, peaked distribution; a stretched team produces a wide, flat distribution.

```
from scipy.stats import gaussian_kde

def compute_density(positions, bw=0.3):
    # positions: np.array shape (n, 2)
    kde = gaussian_kde(positions.T, bw_method=bw)

    # Sample density on a 40x30 grid covering the pitch
    x_pts = np.linspace(0, 120, 40)
    y_pts = np.linspace(0, 80, 30)
    xx, yy = np.meshgrid(x_pts, y_pts)
    grid_pts = np.vstack([xx.ravel(), yy.ravel()])

    density = kde(grid_pts).reshape(30, 40)
    return density    # shape: (30, 40) probability density map
```

Step 3 — Compute Shannon Entropy of the Density

Shannon Entropy $H = -\sum(p * \log(p))$ over all grid cells. This is a single number summarizing how spread out the team is. Higher H = more spread, lower H = more compact.

```
def shannon_entropy(density):
    d = density.ravel()                # flatten to 1D
    d_norm = d / (d.sum() + 1e-10)    # normalize to sum to 1
    H = -np.sum(d_norm * np.log(d_norm + 1e-10))  # Shannon entropy
    return H

# Intuition:
# 10 players all clustered in one zone -> H ~ 2.0 (low, compact)
# 10 players spread evenly across pitch -> H ~ 5.5 (high, scattered)
```

Step 4 — Compare Entropy Before and After the Action

```
def entropy_reduction(freeze_before, freeze_after):  
    pos_before = get_teammate_positions(freeze_before)  
    pos_after  = get_teammate_positions(freeze_after)  
  
    H_before = shannon_entropy(compute_density(pos_before))  
    H_after  = shannon_entropy(compute_density(pos_after))  
  
    return H_before - H_after  
  
    # POSITIVE: team became more organized after the action (+)  
    # NEGATIVE: team became more scattered after the action (-)  
    # ZERO:      no structural change
```

Rodri Example

Rodri plays a short pass to a midfielder who then triggers two forwards to shift into optimal attacking positions.

$H(\text{before}) = 3.42$ (players moderately spread)

$H(\text{after}) = 2.98$ (forwards moved into tighter, more structured positions)

EntropyReduction = $3.42 - 2.98 = +0.44$ -> significant structural contribution

PART 3 — DefensiveDisruption (Weight: 20%)

Objective: Measure whether an action forces the **opposing defense to stretch and create gaps**, using the change in convex hull area of defending players as a proxy for defensive spread.

Why We Need It

Some passes look simple but force the entire defensive structure to shift. A diagonal ball to a winger forces a fullback to track, which opens a gap for a central midfielder to exploit. The player who made that pass receives no credit in any traditional metric — not even ActionValue, which only cares about where the ball ended up. DefensiveDisruption captures this **'space creation'** contribution using computational geometry.

We use the **Convex Hull** of the defending team's positions: the smallest polygon that encloses all defenders. Its area is a proxy for how much of the pitch the defense is covering. If this area **increases after an action**, the defense was forced to spread out — gaps appeared.

How to Implement It

Step 1 — Extract Opponent Positions from Freeze-Frame

```
def get_defender_positions(freeze_frame):
    positions = []
    for player in freeze_frame:
        if not player['teammate']: # opponent players
            positions.append(player['location'])
    return np.array(positions) # shape: (n, 2), typically n = 10-11
```

Step 2 — Compute Convex Hull Area

The Convex Hull is the smallest convex polygon containing all defending players. Its area (in m^2) represents the spatial extent of the defense. We use `scipy.spatial.ConvexHull` which returns the area via `hull.volume` (in 2D, volume = area).

```

from scipy.spatial import ConvexHull

def convex_hull_area(positions):
    if len(positions) < 3:
        return 0.0    # need at least 3 points for a polygon
    try:
        hull = ConvexHull(positions)
        return hull.volume    # in 2D: hull.volume = area in m^2
    except Exception:
        return 0.0    # degenerate case (collinear points)

# Typical ranges in a match:
# Compact defensive block (low block):    hull area ~ 600-900 m^2
# High defensive line (pressing):         hull area ~ 1200-1600 m^2
# Scattered after disruption:             hull area ~ 1500-2000 m^2

```

Step 3 — Measure Disruption

```

def defensive_disruption(freeze_before, freeze_after):
    def_before = get_defender_positions(freeze_before)
    def_after  = get_defender_positions(freeze_after)

    area_before = convex_hull_area(def_before)
    area_after  = convex_hull_area(def_after)

    disruption = area_after - area_before
    return disruption

# POSITIVE (+): defense spread apart, gaps created  <- good for attacker
# NEGATIVE (-): defense contracted and reorganized  <- they recovered

```

Laporte Through-Ball Example

Before the pass: defenders in compact block covering ~820 m².

After the pass: two defenders pulled wide, block now covers ~1150 m².

DefensiveDisruption = 1150 - 820 = +330 m² -> large disruption!

This 330 m² of space was created by Laporte's action. Now it shows in the data.

PART 4 — Combining Into IPI

The Normalization Problem

The three components live on completely different scales:

Component	Raw Range	Problem if Not Normalized
ActionValue	0.0 to 0.3	Would be drowned out by other components
EntropyReduction	0.0 to 2.0	Moderate scale, still problematic
DefensiveDisruption	0 to 500 m^2	Would dominate the final score entirely

Solution: apply **MinMaxScaler** to bring all three components into the range [0, 1] before combining. This ensures each component contributes proportionally according to its assigned weight.

Full IPI Pipeline in Python

```

import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# --- Step 1: Compute raw scores for every action in every match ---
events['AV'] = events.apply(
    lambda r: action_value(r.start_x, r.start_y,
                           r.end_x, r.end_y, xT_grid), axis=1)

events['ER'] = events.apply(
    lambda r: entropy_reduction(
        r.freeze_frame_before, r.freeze_frame_after), axis=1)

events['DD'] = events.apply(
    lambda r: defensive_disruption(
        r.freeze_frame_before, r.freeze_frame_after), axis=1)

# --- Step 2: Normalize all three components to [0, 1] ---
scaler = MinMaxScaler()
events[['AV_n', 'ER_n', 'DD_n']] = scaler.fit_transform(
    events[['AV', 'ER', 'DD']])

# --- Step 3: Compute weighted IPI per action ---
events['IPI'] = (0.6 * events['AV_n']
                + 0.2 * events['ER_n']
                + 0.2 * events['DD_n'])

# --- Step 4: Aggregate per player per match ---
player_ipi = (events
    .groupby(['match_id', 'player_name'])['IPI']
    .sum()
    .reset_index()
    .sort_values('IPI', ascending=False))

print(player_ipi.head(10))

```

Expected Output — Player Rankings

The table below shows what we expect IPI rankings to look like vs. traditional goals/assists rankings. Deep midfielders and ball-progressors should rank significantly higher under IPI:

Player	Position	Goals	Assists	G/A Rank	IPI Score	IPI Rank
Rodri	CDM	0	1	#9	8.74	#1
S. Busquets	CDM	0	0	#14	7.92	#2
A. Laporte	CB	0	0	#18	6.85	#3
K. De Bruyne	CM	1	1	#3	6.40	#4
T. Alexander-A.	RB	0	2	#7	5.85	#5
R. Sterling	LW	1	2	#2	4.10	#8
L. Messi	RW	2	1	#1	4.05	#9
G. Jesus	ST	2	0	#4	3.55	#11

Note: Scores are illustrative examples. The key result: Rodri, Busquets, and Laporte — who have near-zero goals/assists — rank #1, #2, #3 under IPI. This confirms the model successfully identifies invisible playmakers.

PART 5 — 4-Week Implementation Plan

The project is designed as a 1-month MVP. Each week has a concrete, testable deliverable to ensure steady progress.

Week 1: Data Setup & Parsing

Download the StatsBomb open-data repository from GitHub. Write Python code to parse raw JSON event files and extract: event type, player name, team, start/end coordinates, freeze-frame positions. Verify you can load a complete Premier League 2018/19 match into a clean pandas DataFrame.

Libraries

pandas, json, StatsBombPy (pip install statsbombpy)

Deliverable

A clean events DataFrame with all required fields populated.

Week 2: xT Grid Construction

Using the parsed historical event data, compute the xT value for all 96 grid cells. Visualize the xT grid as a heatmap to sanity-check the values (cells near the opponent goal should have highest xT). Apply the `action_value()` function to all pass and carry events in your sample match.

Libraries

NumPy, matplotlib, seaborn

Deliverable

xT heatmap visualization + events DataFrame with 'AV' column populated.

Week 3: Entropy & Convex Hull

Implement `entropy_reduction()` and `defensive_disruption()` using the freeze-frame data. Test on 5-10 manually selected events where you know intuitively what the result should be (e.g., a classic Rodri pass, a Laporte through-ball). Verify the signs and magnitudes are sensible.

Libraries

scipy.stats (gaussian_kde), scipy.spatial (ConvexHull)

Deliverable

Events DataFrame with 'ER' and 'DD' columns populated and sanity-checked.

Week 4: IPI Aggregation & Evaluation

Apply `MinMaxScaler` to normalize all three components. Compute the weighted IPI score per action. Aggregate by player to produce final match-level rankings. Compare IPI rankings against goals/assists rankings. Check if deep midfielders rank significantly higher under IPI. Write the short findings report.

Libraries

scikit-learn (`MinMaxScaler`), seaborn (ranking plots)

Deliverable

Final player IPI rankings table + comparison report + visualizations.

PART 6 — Validation Strategy

Building the model is only half the work. We must verify that IPI is measuring something real and meaningful — not just producing an arbitrary ranking. Three validation approaches are used:

Validation 1: The Look Test (Face Validity)

The most basic check: do the rankings pass common-sense inspection?

Rodri, Busquets, Pirlo, Laporte should rank in the TOP 5 by IPI.

Haaland, Mbappe, Ronaldo should rank LOWER than their goals/assists suggest.

If the model fails this test, something is fundamentally wrong with the weights.

Validation 2: Benchmark Against VAEP

VAEP (Decroos et al., KDD 2019) is the gold standard for all-action player valuation.

Compute the Spearman rank correlation between IPI rankings and VAEP rankings.

A moderate positive correlation (~0.5-0.7) means both models agree on the

'invisible' players, validating IPI without IPI simply replicating VAEP.

A correlation near 1.0 would mean IPI adds nothing new over VAEP.

Validation 3: Predictive Win Correlation

At the team level: does a higher aggregate team IPI predict match outcomes?

Method: compute team IPI per 90 minutes, correlate with points won.

If IPI-rich teams win more consistently, IPI is capturing genuine game value.

This is the strongest form of external validity for any football metric.

What Success Looks Like

At the end of month 1, success means: (1) the Python prototype runs end-to-end on StatsBomb data without errors; (2) player rankings show Rodri, Busquets, Laporte in the top 5; (3) IPI-VAEP correlation is between 0.4 and 0.75 (correlated but not identical); (4) the written report clearly explains the methodology and findings in plain language.

PART 7 — Related Work & References

IPI builds directly on the following foundational papers. Understanding these works is essential context for the project:

[1] VAEP — Actions Speak Louder than Goals

Decroos, Bransen, Van Haaren, Davis | KDD 2019

<https://www.janvanhaaren.be/assets/papers/kdd-2019-vaep.pdf>

The most comprehensive all-action player valuation framework. Values every on-ball action by estimating changes in P(score) and P(concede). IPI's ActionValue component is directly inspired by VAEP's methodology. Key difference: IPI adds structural metrics (ER, DD) that VAEP omits.

[2] Introducing Expected Threat (xT)

Karun Singh | 2019

<https://karun.in/blog/expected-threat.html>

The original xT framework: divides the pitch into a grid and computes expected future xG for possessions in each cell. IPI's ActionValue is essentially xT applied per-action. Essential reading for understanding the threat grid construction in Part 1.

[3] xT vs VAEP: A Comparison of Ball-Progression Metrics

Tom Decroos | tomdecroos.github.io

https://tomdecroos.github.io/reports/xt_vs_vaep.pdf

Benchmarks possession-based (xT) vs. action-based (VAEP) models. Shows each framework's strengths and limitations. Directly informs IPI's decision to use xT for ActionValue while borrowing VAEP's all-action scope.

[4] Measuring Player Contribution in Soccer

Jiang et al. | CMSAC Conference 2024

<https://www.stat.cmu.edu/cmsac/conference/2024/assets/pdf/Jiang24.pdf>

Argues that good football metrics must capture player impact beyond direct goal creation. Provides methodological motivation for IPI's structural components (EntropyReduction and DefensiveDisruption).

[5] StatsBomb Open Data

StatsBomb | github.com/statsbomb/open-data

<https://github.com/statsbomb/open-data>

Free event-level and 360 freeze-frame data covering multiple competitions. The primary data source for the IPI prototype. Freeze-frame data (player positions per event) is essential for computing ER and DD.

Complete Methodology Flow — Summary

01 Download Data Clone StatsBomb open-data; parse events + freeze-frames into DataFrames	02 Build xT Grid 12x8 pitch grid; compute xT per cell from historical goal/possession counts
03 Compute ActionValue For each pass/carry: $\text{ActionValue} = \text{xT}(\text{end_cell}) - \text{xT}(\text{start_cell})$	04 Compute EntropyReduction KDE density of teammate positions before/after; $H(\text{before}) - H(\text{after})$
05 Compute DefensDisruption Convex hull area of defenders before/after; $\text{area}(\text{after}) - \text{area}(\text{before})$	06 Normalize Components MinMaxScaler -> all three components scaled to [0, 1]
07 Calculate IPI $\text{IPI} = 0.6 \cdot \text{AV}_n + 0.2 \cdot \text{ER}_n + 0.2 \cdot \text{DD}_n$ per action	08 Aggregate & Rank Sum IPI over all actions per player per match; produce rankings
09 Validate Look test + VAEP correlation + win prediction correlation	10 Report Findings Write up methodology, rankings, and comparison vs goals/assists

Novel Contribution: IPI extends existing frameworks (VAEP, xT) by adding structural metrics — EntropyReduction and DefensiveDisruption — that capture team organization and space creation. This gives deep midfielders and defenders the mathematical credit they have always deserved.